



GUIA PRÁTICA DE ESTUDO

IOI · ICPC · Olimpíadas de Informática

Resumo das Aulas de Teoria de Grafos para Programação Competitiva

Teoria · Algoritmos · Implementações em C++

Prof. Leovel Camilo Neto

Material de apoio para estudantes

Estruturas de Dados · Travessias DFS/BFS · Conectividade · Ordenação Topológica · Árvores de Cobertura Mínima · Caminhos Mínimos · Circuitos Eulerianos

Índice

Representações Estruturais de Grafos e Complexidade Computacional.....	4
Matriz de Adjacência	5
Lista de Adjacência	6
Lista de Arestas.....	7
Pesquisa em Profundidade e Pesquisa em Largura	8
Pesquisa em Profundidade (Depth First Search)	8
Pesquisa em Largura (Breadth First Search)	9
Componentes Conexas e Difusão em Grelhas Implícitas	11
Identificação de Componentes Conexas em Grafos Não Dirigidos	11
Difusão em Grelhas Bidimensionais Implícitas (Flood Fill)	12
Ordenação Topológica em Grafos Dirigidos Acíclicos.....	13
Verificação de Bipartição e Detecção de Ciclos	16
Verificação de Grafos Bipartidos (2-Colorabilidade)	16
Classificação de Arestas e Detecção de Ciclos em Grafos Dirigidos	17
Pontos de Articulação e Pontes em Grafos Não Dirigidos	19
Componentes Fortemente Conexas em Grafos Dirigidos	21

Árvores de Cobertura Mínima: Teoria e Algoritmos Clássicos	23
Algoritmo de Kruskal.....	23
Algoritmo de Prim	25
Aplicações Especializadas de Árvores de Cobertura Mínima	27
Caminhos Mínimos de Fonte Única	29
Algoritmo de Dijkstra	29
Algoritmo de Bellman-Ford	31
Todos os Pares de Caminhos Mínimos.....	33
Grafos Especiais e Circuitos Eulerianos.....	35
Quadro Sintético de Decisão Algorítmica em Competição.....	37

A resolução eficaz de problemas no âmbito das Olimpíadas Internacionais de Informática (IOI) e do Concurso Internacional de Programação Universitária (ICPC) exige o domínio da modelação abstracta de dados através de grafos, aliada à selecção rigorosa de algoritmos com limites estritos de complexidade temporal e espacial. Um grafo $G = (V, E)$ é definido formalmente como um par ordenado composto por um conjunto de vértices (ou nós) V e um conjunto de arestas (ou arcos) $E \subseteq V \times V$, representando relações binárias directas entre os elementos de V . A correcta formulação de um problema competitivo depende, em primeira instância, da identificação das propriedades estruturais de G : se as arestas possuem orientação (grafos dirigidos versus grafos não dirigidos), se possuem pesos associados ($w: E \rightarrow \mathbb{R}$), se existem restrições de conectividade ou se a topologia subjacente respeita categorias especiais como árvores, grafos acíclicos dirigidos e grafos bipartidos.

Representações Estruturais de Grafos e Complexidade Computacional

A selecção da estrutura de dados para representar um grafo determina directamente a complexidade assintótica das operações de consulta e travessia subsequentes. As três representações fundamentais utilizadas em programação competitiva são a Matriz de Adjacência (*Adjacency Matrix* — AM), a Lista de Adjacência (*Adjacency List* — AL) e a Lista de Arestas (*Edge List* — EL).

Estrutura de Dados	Complexidade Espacial	Adição de Aresta	Consulta de Existência (u, v)	Iteração sobre Vizinhos de u	Cenário Típico no ICPC / IOI
Matriz de Adjacência (AM)	$O(V^2)$	$O(1)$	$O(1)$	$O(V)$	Grafos densos ($E \approx V^2$), $V \leq 1000$, ou algoritmos que requerem operações de matriz.

Lista de Adjacência (AL)	$O(V + E)$	$O(1)$ amortizado	$O(\text{grau}(u))$	$O(\text{grau}(u))$	Padrão óptimo para grafos esparsos ($V, E \leq 2 \times 10^5$).
Lista de Arestas (EL)	$O(E)$	$O(1)$	$O(E)$	$O(E)$	Algoritmo de Kruskal e rotinas de ordenação global de arestas.

Matriz de Adjacência

A Matriz de Adjacência armazena uma tabela bidimensional de dimensões $|V| \times |V|$, onde a posição $AM[u][v]$ registra a existência da aresta ou a respectiva ponderação quantitativa de peso. O seu emprego em contexto competitivo fica restrito a instâncias onde $|V| \leq 1000$, dado que valores superiores provocam a ultrapassagem catastrófica dos limites de memória física da plataforma de submissão. A matriz é inicializada com valores convencionais de infinito nas posições desconectadas e com zero na diagonal principal.

C++ | Matriz de Adjacência

```
#include <vector>

const int INF = 1e9;
int V = 5;
std::vector<std::vector<int>>> AM(V, std::vector<int>(V, INF));

void adicionarArestaAM(int u, int v, int peso, bool dirigido = false) {
    AM[u][v] = peso;
    if (!dirigido) {
        AM[v][u] = peso;
    }
}
```

```
}
```

Lista de Adjacência

A Lista de Adjacência associa a cada vértice u um contentor dinâmico contendo pares ordenados (v, w) , que especificam o nó adjacente e o respectivo peso da aresta. Em linguagem C++, a estrutura canónica é implementada através de `std::vector<std::vector<std::pair<int, int>>>`, a qual optimiza o consumo espacial estritamente para $O(V + E)$ e permite iterar exclusivamente sobre os vizinhos reais de cada vértice durante as travessias.

C++ | Lista de Adjacência

```
#include <vector>
#include <utility>

using ii = std::pair<int, int>;
using vii = std::vector<ii>;

int V = 5;
std::vector<vii> AL(V);

void adicionarArestaAL(int u, int v, int w, bool dirigido = false) {
    AL[u].push_back({v, w});
    if (!dirigido) {
        AL[v].push_back({u, w});
    }
}
```

Lista de Arestas

A Lista de Arestas armazena a totalidade das arestas de forma desarticulada num vector unidimensional contendo tuplos compostos por peso, origem e destino. Esta representação é desprovida de capacidade para responder de forma imediata à vizinhança de um nó isolado, mas constitui a estrutura ideal para algoritmos cujo passo inicial assenta na ordenação directa do conjunto E , tais como o algoritmo de Kruskal para árvores de cobertura mínima.

C++ | Lista de Arestas

```
#include <vector>
#include <tuple>

using iii = std::tuple<int, int, int>;
std::vector<iii> EL;

void adicionarArestaEL(int u, int v, int w) {
    EL.emplace_back(w, u, v);
}
```

Pesquisa em Profundidade e Pesquisa em Largura

Pesquisa em Profundidade (Depth First Search)

A Pesquisa em Profundidade (DFS) progride através da exploração linear de ramos individuais até ao vértice mais profundo exequível antes de retroceder sistematicamente através de mecanismos de retorno recursivo. O algoritmo utiliza a pilha de chamadas de execução do sistema para conservar o rasto de exploração. Para evitar divergências em ciclos infinitos, a DFS mantém um vector de estados de visitação. A complexidade temporal sobre listas de adjacência é linear em $O(V + E)$.

C++ | Pesquisa em Profundidade (DFS)

```
#include <vector>
#include <utility>

using ii = std::pair<int, int>;
using vii = std::vector<ii>;
using vi = std::vector<int>;

enum { NAO_VISITADO = -1, VISITADO = 1 };

std::vector<vii> AL;
vi dfs_num;

void dfs(int u) {
    dfs_num[u] = VISITADO;
    for (const auto &[v, w] : AL[u]) {
        if (dfs_num[v] == NAO_VISITADO) {
            dfs(v);
        }
    }
}
```

Pesquisa em Largura (Breadth First Search)

A Pesquisa em Largura (BFS) expande a sua fronteira de descoberta de forma concêntrica, visitando todos os vértices situados a k arestas da origem antes de avançar para aqueles localizados a distância $k + 1$. Esta ordem de travessia requer a utilização de uma fila com política de primeiro a entrar, primeiro a sair (std::queue). Em virtude desta propagação por camadas uniformes, a BFS calcula o caminho mínimo em número de arestas em qualquer grafo não ponderado no tempo rigoroso de $O(V + E)$.

C++ | Pesquisa em Largura (BFS)

```
#include <vector>
#include <queue>
#include <utility>

using ii = std::pair<int, int>;
using vii = std::vector<ii>;
using vi = std::vector<int>;

const int INF = 1e9;
std::vector<vii> AL;
vi dist;

void bfs(int s, int V) {
    dist.assign(V, INF);
    std::queue<int> q;
    dist[s] = 0;
    q.push(s);

    while (!q.empty()) {
        int u = q.front();
        q.pop();
```

```
    for (const auto &[v, w] : AL[u]) {  
        if (dist[v] == INF) {  
            dist[v] = dist[u] + 1;  
            q.push(v);  
        }  
    }  
}
```

Componentes Conexas e Difusão em Grelhas Implícitas

Identificação de Componentes Conexas em Grafos Não Dirigidos

Uma componente conexa é definida como um subgrafo conexo maximal no qual todos os pares de vértices se encontram mutuamente conectados por via de caminhos simples. A determinação de todas as componentes conexas de um grafo não dirigido é efectuada através de uma iteração sequencial sobre os índices de 0 a $V - 1$. Caso um dado nó se encontre no estado não visitado, dispara-se uma travessia completa por DFS ou BFS, a qual marcará a totalidade dos elementos pertencentes à sua componente e incrementará a contagem global.

C++ | Contagem de Componentes Conexas

```
#include <vector>

std::vector<vii> AL;
vi dfs_num;

int contarComponentesConexas(int V) {
    dfs_num.assign(V, NAO_VISITADO);
    int totalCC = 0;
    for (int u = 0; u < V; ++u) {
        if (dfs_num[u] == NAO_VISITADO) {
            ++totalCC;
            dfs(u);
        }
    }
    return totalCC;
}
```

Difusão em Grelhas Bidimensionais Implícitas (Flood Fill)

Inúmeros problemas competitivos apresentam o grafo de forma indirecta sob o formato de uma matriz bidimensional de dimensões $R \times C$. Cada elemento (r, c) desempenha a função de vértice, estabelecendo adjacências funcionais imediatas com as células vizinhas em quatro direcções ortogonais ou em oito direcções incluindo diagonais. O algoritmo de preenchimento por difusão (flood fill) opera a travessia directamente sobre a matriz sem recorrer à alocação de uma lista de adjacência, validando fronteiras de grelha e condições de transição cromática em tempo $O(R \cdot C)$.

C++ | Flood Fill em Grelha 2D

```
#include <vector>

int R, C;
std::vector<std::vector<char>> grelha;

const int dr[] = { -1, 0, 1, 0 };
const int dc[] = { 0, 1, 0, -1 };

int floodFill(int r, int c, char corOriginal, char novaCor) {
    if (r < 0 || r >= R || c < 0 || c >= C) return 0;
    if (grelha[r][c] != corOriginal) return 0;

    grelha[r][c] = novaCor;
    int tamanhoComponente = 1;

    for (int d = 0; d < 4; ++d) {
        tamanhoComponente += floodFill(r + dr[d], c + dc[d], corOriginal, novaCor);
    }
    return tamanhoComponente;
}
```

Ordenação Topológica em Grafos Dirigidos Acíclicos

A ordenação topológica de um grafo dirigido acíclico (Directed Acyclic Graph — DAG) traduz-se numa disposição linear de todos os seus vértices na qual, para cada aresta dirigida $(u, v) \in E$, o nó de origem u surge obrigatoriamente antes do nó de destino v na sequência final. A existência de pelo menos uma ordenação topológica válida constitui condição necessária e suficiente para assegurar que a estrutura do grafo é rigorosamente acíclica.

Metodologia de Ordenação	Estrutura de Controlo	Estratégia de Processamento	Deteção de Ciclos no Grafo
Pesquisa em Profundidade (DFS)	Pilha de Recursão	Adiciona o nó no instante de conclusão e inverte a lista global.	Requer monitorização adicional de arestas de retorno.
Algoritmo de Kahn	Fila de Prioridade ou FIFO	Elimina sucessivamente nós com grau de entrada nulo.	Detecta ciclos caso a ordenação final contenha menos de V nós.

Na abordagem assente em DFS, explora-se a propriedade segundo a qual um vértice conclui a sua função recursiva unicamente quando todos os caminhos que dele partem foram esgotados. Ao inserir o vértice numa lista global imediatamente antes do retorno da função e ao inverter essa mesma lista após a exploração exhaustiva de todos os nós, obtém-se a ordenação correcta em tempo linear $O(V + E)$.

C++ | Ordenação Topológica por DFS

```
#include <vector>
```

```
#include <algorithm>

std::vector<vii> AL;
vi dfs_num;
vi ordemTopologica;

void toposortDFS(int u) {
    dfs_num[u] = VISITADO;
    for (const auto &[v, w] : AL[u]) {
        if (dfs_num[v] == NAO_VISITADO) {
            toposortDFS(v);
        }
    }
    ordemTopologica.push_back(u);
}

std::vector<int> executarToposort(int V) {
    dfs_num.assign(V, NAO_VISITADO);
    ordemTopologica.clear();
    for (int u = 0; u < V; ++u) {
        if (dfs_num[u] == NAO_VISITADO) {
            toposortDFS(u);
        }
    }
    std::reverse(ordemTopologica.begin(), ordemTopologica.end());
    return ordemTopologica;
}
```

O algoritmo de Kahn calcula inicialmente os graus de entrada de todos os vértices e enfileira aqueles que possuem grau zero. À medida que estes nós são consumidos, decrementam-se os graus dos seus vizinhos directos, inserindo na fila qualquer novo vértice cujo grau atinja zero. A utilização de uma fila de prioridade com ordenação léxica nesta variante permite determinar a menor ordenação topológica lexicográfica viável.

C++ | Algoritmo de Kahn

```
#include <vector>
#include <queue>

std::vector<int> kahnToposort(int V, const std::vector<vii> &AL) {
    vi grauEntrada(V, 0);
    for (int u = 0; u < V; ++u) {
        for (const auto &[v, w] : AL[u]) {
            grauEntrada[v]++;
        }
    }

    std::queue<int> fila;
    for (int u = 0; u < V; ++u) {
        if (grauEntrada[u] == 0) {
            fila.push(u);
        }
    }

    std::vector<int> resultado;
    while (!fila.empty()) {
        int u = fila.front();
        fila.pop();
        resultado.push_back(u);

        for (const auto &[v, w] : AL[u]) {
            if (--grauEntrada[v] == 0) {
                fila.push(v);
            }
        }
    }
    return resultado;
}
```

Verificação de Bipartição e Detecção de Ciclos

Verificação de Grafos Bipartidos (2-Colorabilidade)

Um grafo não dirigido G é bipartido se o seu conjunto de vértices admitir uma partição estrita em dois subconjuntos disjuntos e independentes V_1 e V_2 , de tal modo que qualquer aresta incidente a um elemento de V_1 a um elemento de V_2 . O teorema de König postula que um grafo é bipartido se e só se não contiver ciclos de comprimento ímpar. A verificação de bipartição processa-se através da tentativa de coloração gulosa com duas cores discretas por intermédio de uma travessia BFS ou DFS. Ao descobrir-se que um vértice adjacente ostenta a mesma cor que o vértice sob análise, atesta-se a inviabilidade da bipartição.

C++ | Verificação de Grafo Bipartido

```
#include <vector>
#include <queue>

bool verificarBipartido(int s, int V, const std::vector<vii> &AL) {
    vi cor(V, -1);
    std::queue<int> q;
    cor[s] = 0;
    q.push(s);

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (const auto &[v, w] : AL[u]) {
            if (cor[v] == -1) {
                cor[v] = 1 - cor[u];
                q.push(v);
            } else if (cor[v] == cor[u]) {
```

```
        return false;
    }
}
return true;
}
```

Classificação de Arestas e Detecção de Ciclos em Grafos Dirigidos

A detecção de ciclos em grafos dirigidos repousa sobre a classificação das arestas geradas pela árvore de cobertura construída pela pesquisa em profundidade. O algoritmo monitoriza a transição de três estados distintos para cada nó: NAO_VISITADO, EM_EXPLORACAO e PROCESSADO.

Natureza da Aresta	Estado do Destino na Transição	Significado Algorítmico
Aresta de Árvore (Tree Edge)	NAO_VISITADO	Descobre e incorpora um novo vértice na árvore de exploração.
Aresta de Retorno (Back Edge)	EM_EXPLORACAO	Conecta o nó actual a um predecessor activo; confirma a existência de um ciclo .
Aresta de Avanço (Forward Edge)	PROCESSADO	Conecta o nó a um dos seus descendentes na árvore que já concluiu a execução.
Aresta Cruzada (Cross Edge)	PROCESSADO	Conecta nós entre ramos distintos sem relação directa de ascendência ou descendência.

A detecção de qualquer aresta orientada para um vértice no estado EM_EXPLORACAO fecha imediatamente um ciclo dirigido, inviabilizando qualquer pretensão de aciclicidade na rede sob teste.

C++ | Detecção de Ciclos em Grafos Dirigidos

```
#include <vector>

enum { NAO_VISITADO = -1, EM_EXPLORACAO = -2, PROCESSADO = -3 };

std::vector<vii> AL;
vi dfs_num;
vi dfs_parent;
bool contemCiclo = false;

void cycleCheck(int u) {
    dfs_num[u] = EM_EXPLORACAO;
    for (const auto &[v, w] : AL[u]) {
        if (dfs_num[v] == NAO_VISITADO) {
            dfs_parent[v] = u;
            cycleCheck(v);
        } else if (dfs_num[v] == EM_EXPLORACAO) {
            contemCiclo = true;
        }
    }
    dfs_num[u] = PROCESSADO;
}
```

Pontos de Articulação e Pontes em Grafos Não Dirigidos

Num grafo não dirigido e conexo, um **ponto de articulação** é um vértice cuja remoção, em conjunto com as arestas que nele incidem, provoca a desconexão do grafo em duas ou mais componentes conexas disjuntas. Uma **ponte** é uma aresta cuja eliminação isolada compromete a conectividade global do grafo. Estes dois elementos desempenham um papel crítico na identificação de estrangulamentos estruturais e pontos únicos de falha em redes de comunicação.

O algoritmo de Tarjan efectua a identificação de todos os pontos de articulação e pontes numa única travessia DFS com complexidade estritamente linear $O(V + E)$. A execução baseia-se na manutenção de dois vectores inteiros designados por `dfs_num` e `dfs_low`. O vector `dfs_num[u]` armazena a ordem temporal de descoberta de cada nó na travessia. O vector `dfs_low[u]` regista o menor identificador temporal `dfs_num` alcançável a partir do conjunto de nós da subárvore enraizada em u , permitindo no máximo o uso de uma aresta de retorno.

A classificação dos pontos de articulação requer a distinção entre a raiz da árvore de profundidade e os vértices internos. A raiz da árvore de profundidade constitui um ponto de articulação se e só se possuir dois ou mais filhos directos na árvore gerada. Um vértice não-raiz u constitui um ponto de articulação se possuir pelo menos um filho v que satisfaça a inequação $\text{dfs_low}[v] \geq \text{dfs_num}[u]$, o que comprova que a subárvore enraizada em v não dispõe de qualquer aresta de retorno para alcançar um antecessor directo de u . De forma homóloga, uma aresta (u, v) constitui uma ponte se e só se verificar a desigualdade estrita $\text{dfs_low}[v] > \text{dfs_num}[u]$.

C++ | Pontos de Articulação e Pontes (Tarjan)

```
#include <vector>
#include <algorithm>
#include <utility>
```

```
using ii = std::pair<int, int>;
using vii = std::vector<ii>;
using vi = std::vector<int>;

std::vector<vii> AL;
vi dfs_num, dfs_low, dfs_parent, articulacao;
int dfsContador, dfsRaiz, filhosRaiz;

void encontrarCortes(int u) {
    dfs_low[u] = dfs_num[u] = dfsContador++;
    for (const auto &[v, w] : AL[u]) {
        if (dfs_num[v] == -1) {
            dfs_parent[v] = u;
            if (u == dfsRaiz) ++filhosRaiz;

            encontrarCortes(v);

            if (dfs_low[v] >= dfs_num[u]) {
                articulacao[u] = 1;
            }
            if (dfs_low[v] > dfs_num[u]) {
                // Aresta (u, v) constitui uma ponte
            }
            dfs_low[u] = std::min(dfs_low[u], dfs_low[v]);
        } else if (v != dfs_parent[u]) {
            dfs_low[u] = std::min(dfs_low[u], dfs_num[v]);
        }
    }
}
```

Componentes Fortemente Conexas em Grafos Dirigidos

Uma **Componente Fortemente Conexa** (*Strongly Connected Component* — SCC) de um grafo dirigido G corresponde a um subgrafo maximal no qual, para qualquer par ordenado de nós (u, v) , existe um caminho dirigido de u para v e um caminho de regresso de v para u . A contracção de cada componente num super-vértice origina o grafo condensado ou grafo de componentes, cuja topologia é forçosamente a de um DAG. Esta decomposição linear viabiliza a aplicação de Programação Dinâmica a redes dirigidas arbitrárias que originalmente continham ciclos.

O algoritmo de Tarjan extrai as SCCs em tempo linear $O(V + E)$ através de uma travessia por DFS complementada por uma pilha explícita que armazena os vértices em curso de análise. À medida que a pesquisa desce na estrutura, os nós são empilhados; o valor `dfs_low[u]` é minimizado relativamente a arestas de retorno e arestas cruzadas cujos destinos ainda permaneçam na pilha. No instante em que o retorno recursivo identifica um nó u cuja condição `dfs_low[u] = dfs_num[u]` é satisfeita, reconhece-se esse nó como raiz da sua componente fortemente conexa. Todos os nós acumulados acima de u na pilha são então retirados, formalizando a partição de uma SCC completa.

C++ | Componentes Fortemente Conexas (Tarjan)

```
#include <vector>
#include <stack>
#include <algorithm>

std::vector<vii> AL;
vi dfs_num, dfs_low, noNaPilha;
std::stack<int> pilha;
int contadorTempo = 0, totalSCC = 0;

void tarjanSCC(int u) {
```

```
dfs_low[u] = dfs_num[u] = contadorTempo++;
pilha.push(u);
noNaPilha[u] = 1;

for (const auto &[v, w] : AL[u]) {
    if (dfs_num[v] == -1) {
        tarjanSCC(v);
        dfs_low[u] = std::min(dfs_low[u], dfs_low[v]);
    } else if (noNaPilha[v]) {
        dfs_low[u] = std::min(dfs_low[u], dfs_num[v]);
    }
}

if (dfs_low[u] == dfs_num[u]) {
    ++totalSCC;
    while (true) {
        int v = pilha.top();
        pilha.pop();
        noNaPilha[v] = 0;
        if (u == v) break;
    }
}
}
```

Árvores de Cobertura Mínima: Teoria e Algoritmos Clássicos

Dado um grafo conexo e ponderado não dirigido $G = (V, E)$, uma **Árvore de Cobertura Mínima** (*Minimum Spanning Tree* — MST) é um subconjunto de arestas $T \subseteq E$ que conecta a totalidade dos vértices $|V|$ sem a formação de ciclos e cujo somatório de pesos $\sum_{e \in T} w(e)$ atinge o menor valor possível. A demonstração de correcção dos algoritmos gulosos de cobertura fundamenta-se no **Teorema do Corte**: dado qualquer corte de vértices $(S, V \setminus S)$ no grafo que não intersecte nenhuma das arestas já seleccionadas, a aresta de peso mínimo que cruza esse corte pertence de forma garantida a uma árvore de cobertura mínima óptima.

Algoritmo	Estrutura de Suporte	Complexidade Temporal	Comportamento Topológico
Kruskal	Union-Find Disjoint Sets	$O(E \log E)$	Agrega componentes isoladas através de ordenação global das arestas.
Prim	Fila de Prioridade Mínima	$O(E \log V)$	Expande continuamente uma única componente conexa a partir da raiz.

Algoritmo de Kruskal

O algoritmo de Kruskal adopta uma perspectiva centrada nas arestas. O processo inicia-se com a ordenação não decrescente de todas as arestas do conjunto E . Em seguida, itera-se sobre a lista ordenada, seleccionando cada aresta que não una vértices já pertencentes à mesma componente conexa. A validação de conectividade e a junção dos subconjuntos disjuntos são coordenadas por uma estrutura *Union-Find Disjoint Sets* (UFDS) dotada de união por ordem de escalão (*union by rank*) e compressão de percurso (path compression), o que garante operações quase lineares regidas pelo inverso da função de Ackermann $\alpha(V)$.

C++ | Algoritmo de Kruskal com Union-Find

```
#include <vector>
#include <tuple>
#include <algorithm>

using iii = std::tuple<int, int, int>;
using vi = std::vector<int>;

class UnionFind {
private:
    vi p, rank;
public:
    UnionFind(int N) {
        p.assign(N, 0); for (int i = 0; i < N; ++i) p[i] = i;
        rank.assign(N, 0);
    }
    int findSet(int i) { return (p[i] == i) ? i : (p[i] = findSet(p[i])); }
    bool isSameSet(int i, int j) { return findSet(i) == findSet(j); }
    void unionSet(int i, int j) {
        if (!isSameSet(i, j)) {
            int x = findSet(i), y = findSet(j);
            if (rank[x] > rank[y]) std::swap(x, y);
            p[x] = y;
            if (rank[x] == rank[y]) ++rank[y];
        }
    }
};

int kruskalMST(int V, std::vector<iii> &EL) {
    std::sort(EL.begin(), EL.end());
    UnionFind UF(V);
    int custoMST = 0, arestasContabilizadas = 0;

    for (const auto &[w, u, v] : EL) {
        if (!UF.isSameSet(u, v)) {
```

```
        custoMST += w;
        UF.unionSet(u, v);
        if (++arestasContabilizadas == V - 1) break;
    }
}
return custoMST;
}
```

Algoritmo de Prim

O algoritmo de Prim desenvolve a árvore através de um crescimento monopolar partindo de um nó arbitrário. O conjunto de vértices é dividido entre os elementos já anexados à árvore e os restantes. Uma fila de prioridade mínima armazena todas as arestas que atravessam este corte. Em cada iteração, extrai-se a aresta de peso mínimo que alcança um vértice ainda não coberto, incorporando-o na solução e inserindo na fila as novas arestas incidentes.

C++ | Algoritmo de Prim

```
#include <vector>
#include <queue>
#include <utility>

using ii = std::pair<int, int>;
using vii = std::vector<ii>;
using vi = std::vector<int>;

std::vector<vii> AL;
vi tomado;
std::priority_queue<ii, std::vector<ii>, std::greater<ii>> pq;

void processarVertice(int u) {
```

```
    tomado[u] = 1;
    for (const auto &[v, w] : AL[u]) {
        if (!tomado[v]) {
            pq.push({w, v});
        }
    }
}

int primMST(int V) {
    tomado.assign(V, 0);
    processarVertice(0);
    int custoMST = 0, arestasContabilizadas = 0;

    while (!pq.empty()) {
        auto [w, u] = pq.top();
        pq.pop();

        if (tomado[u]) continue;

        custoMST += w;
        processarVertice(u);
        if (++arestasContabilizadas == V - 1) break;
    }
    return custoMST;
}
```

Aplicações Especializadas de Árvores de Cobertura Mínima

A teoria de cobertura mínima estende-se a diversas classes de problemas práticos de modelação geométrica e optimização combinatória.

A **Segunda Melhor Árvore de Cobertura Mínima** procura determinar a árvore de suporte que possui o menor custo possível estritamente superior ou igual à MST original. O procedimento analítico assenta na construção prévia da MST T . Qualquer aresta externa $(u, v) \notin T$, quando introduzida em T , produz exactamente um ciclo simples. A restauração da topologia de árvore faz-se pela eliminação da aresta de peso estritamente máximo contida nesse ciclo. Para obter a segunda melhor árvore, avalia-se o incremento de custo gerado por essa troca para todas as arestas externas a T , identificando a menor variação positiva. O cálculo eficiente da aresta máxima no caminho que une u a v ao longo da árvore efectua-se em $O(\log V)$ recorrendo à pré-computação do menor antepassado comum (*Lowest Common Ancestor* — *LCA*) com agregação de máximos por exponenciação binária (binary lifting).

A **Árvore de Cobertura de Gargalo Mínimo** (*Minimum Bottleneck Spanning Tree*) exige a minimização do peso da aresta individual mais pesada que compõe a cobertura da rede. Por força directa do Teorema do Corte, qualquer árvore de cobertura mínima gerada pelos algoritmos de Kruskal ou Prim constitui, por si só, uma árvore de gargalo mínimo ideal. A resolução deste problema decorre assim da execução directa dos algoritmos clássicos de MST sem modificações adicionais.

A **Determinação de Florestas para Agrupamento de Dados** (*Clustering*) consiste na partição dos vértices em exactamente K agrupamentos maximamente segregados entre si. Esta formulação é resolvida através de uma terminação precoce do algoritmo de Kruskal. Ao iniciar o algoritmo sobre uma floresta de $|V|$ componentes desconectadas, cada junção gulosa decrementa o total de componentes em uma unidade. Interrompe-se o laço de Kruskal precisamente no momento em que a estrutura atinge K componentes

conexas disjuntas. O peso da próxima aresta que uniria duas componentes distintas reflecte rigorosamente a distância máxima de separação entre os agrupamentos gerados.

Caminhos Mínimos de Fonte Única

O problema dos Caminhos Mínimos de Fonte Única (*Single-Source Shortest Paths* — SSSP) determina a menor soma acumulada de pesos entre um nó de origem fixo $s \in V$ e todos os restantes nós $v \in V$. A natureza matemática dos pesos das arestas e a existência de ciclos orientam a escolha do algoritmo adequado.

Algoritmo	Restrição de Pesos	Topologia do Grafo	Complexidade Temporal	Princípio de Operação
Pesquisa em Largura (BFS)	Pesos uniformes ou unitários	Grafos arbitrários	$O(V + E)$	Travessia concêntrica em níveis de distância uniforme.
Algoritmo de Dijkstra	Pesos não-negativos ($w \geq 0$)	Grafos arbitrários	$O((V + E) \log V)$	Relaxamento guloso guiado por fila de prioridade.
Algoritmo de Bellman-Ford	Pesos arbitrários (suporta negativos)	Grafos sem ciclos negativos	$O(V \cdot E)$	Rondas sistemáticas de relaxamento em todas as arestas.
Programação Dinâmica em DAG	Pesos arbitrários	Grafos acíclicos dirigidos	$O(V + E)$	Relaxamento estruturado segundo a ordenação topológica.

Algoritmo de Dijkstra

O algoritmo de Dijkstra apoia-se na constatação de que caminhos com custos exclusivamente não-negativos produzem sequências monotônicas não decrescentes ao longo do percurso. Mantém-se uma estimativa de limite superior no vector *dist* e utiliza-se uma fila de prioridade mínima que extrai iterativamente o nó u de menor distância provisória. As entradas da fila que possuem distâncias

superiores à melhor marca registada são imediatamente descartadas. Ao relaxar uma aresta adjacente (u, v) de peso w , a condição $\text{dist}[u] + w < \text{dist}[v]$ permite actualizar a estimativa de v e propagar o novo valor para a fila de prioridade.

C++ | Algoritmo de Dijkstra

```
#include <vector>
#include <queue>
#include <utility>

using ii = std::pair<int, int>;
using vii = std::vector<ii>;
using vi = std::vector<int>;

const int INF = 1e9;
std::vector<vii> AL;
vi dist;

void dijkstra(int s, int V) {
    dist.assign(V, INF);
    dist[s] = 0;
    std::priority_queue<ii, std::vector<ii>, std::greater<ii>> pq;
    pq.push({0, s});

    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();

        if (d > dist[u]) continue;

        for (const auto &[v, w] : AL[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}
```

```

    }
  }
}

```

Algoritmo de Bellman-Ford

A presença de arestas com pesos negativos quebra o princípio guloso do algoritmo de Dijkstra, dado que um percurso aparentemente subóptimo pode vir a ser compensado por uma aresta negativa subsequente. O algoritmo de Bellman-Ford contorna esta falha ao executar $|V| - 1$ iterações de relaxamento sobre a totalidade do conjunto E . Dado que qualquer caminho simples de custo mínimo contém no máximo $|V| - 1$ arestas, este número de passagens assegura a convergência das distâncias de todos os nós alcançáveis. A execução de uma V -ésima passagem sobre as arestas permite a detecção inequívoca de **ciclos de peso negativo**: se for possível relaxar qualquer aresta nesta passagem adicional, confirma-se que o vértice pertence ou é alcançável por um ciclo negativo, gerando distâncias ilimitadamente decrescentes para menos infinito.

C++ | Algoritmo de Bellman-Ford

```

#include <vector>
#include <tuple>

using iii = std::tuple<int, int, int>;
using vi = std::vector<int>;

const int INF = 1e9;
vi dist;

bool bellmanFord(int s, int V, const std::vector<iii> &arestas) {
    dist.assign(V, INF);

```

```
dist[s] = 0;

for (int i = 0; i < V - 1; ++i) {
    for (const auto &[u, v, w] : arestas) {
        if (dist[u] != INF && dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
        }
    }
}

for (const auto &[u, v, w] : arestas) {
    if (dist[u] != INF && dist[u] + w < dist[v]) {
        return false;
    }
}
return true;
}
```

Todos os Pares de Caminhos Mínimos

O problema de determinar caminhos mínimos entre todos os pares arbitrários de vértices (*All-Pairs Shortest Paths* — APSP) encontra no algoritmo de **Floyd-Warshall** a sua solução canónica. Construído sob o paradigma da Programação Dinâmica, o algoritmo baseia-se na formulação de subproblemas estruturados por uma variável intermediária k . A relação de recorrência define que o caminho óptimo entre a origem u e o destino v , admitindo exclusivamente nós intermediários pertencentes ao subconjunto $\{0, 1, \dots, k\}$, corresponde ao mínimo entre o valor já apurado na etapa $k - 1$ e a combinação do caminho de u até k com o caminho de k até v :

$$D^{(k)}[u][v] = \min(D^{(k-1)}[u][v], D^{(k-1)}[u][k] + D^{(k-1)}[k][v])$$

A optimização do espaço computacional permite projectar a tabela tridimensional directamente sobre a matriz de adjacência bidimensional original. A complexidade temporal de $O(V^3)$ é executada através de três laços encadeados. A variável k deve obrigatoriamente constituir o laço mais externo, de modo a preservar as propriedades de subestrutura óptima da Programação Dinâmica.

C++ | Algoritmo de Floyd-Warshall

```
#include <vector>
#include <algorithm>

const int INF = 1e9;
const int MAX_V = 450;
int AM[MAX_V][MAX_V];

void floydWarshall(int V) {
    for (int k = 0; k < V; ++k) {
```

```
for (int u = 0; u < V; ++u) {
    for (int v = 0; v < V; ++v) {
        if (AM[u][k] < INF && AM[k][v] < INF) {
            AM[u][v] = std::min(AM[u][v], AM[u][k] + AM[k][v]);
        }
    }
}
```

O algoritmo de Floyd-Warshall adapta-se igualmente ao cálculo do Fecho Transitivo de Warshall através da substituição das operações aritméticas por operadores lógicos ($AM[u][v] \mid= (AM[u][k] \ \& \ AM[k][v])$), bem como à identificação de ciclos de peso negativo, assinalados pela existência de valores negativos na diagonal principal ($AM[i][i] < 0$) após o processamento completo.

Grafos Especiais e Circuitos Eulerianos

Determinadas topologias de grafos admitem soluções algorítmicas de complexidade substancialmente inferior à das formulações gerais.

Família Estrutural	Propriedades Canônicas Relevantes	Aplicação Algorítmica Específica
Árvore	Grafo conexo, acíclico, com $ E = V - 1$ arestas.	Cálculo do diâmetro em $O(V)$ através de duas travessias consecutivas.
Grafo Acíclico Dirigido (DAG)	Ausência de ciclos orientados; admite ordenação linear.	Determinação de caminhos mínimos e máximos em $O(V + E)$.
Grafo Euleriano	Possui percurso fechado ou aberto que visita cada aresta uma única vez.	Extracção do circuito em tempo linear $O(V + E)$ por Hierholzer.

O diâmetro de uma árvore não ponderada ou ponderada com pesos não-negativos corresponde à distância máxima mensurável entre qualquer par de nós. Esta métrica é determinada em tempo estritamente linear através da execução de duas pesquisas BFS ou DFS sucessivas. A primeira pesquisa parte de um nó arbitrário x com o intuito de localizar o vértice y mais distante de x . A segunda pesquisa parte de y para encontrar o vértice z mais afastado deste. A distância final apurada entre y e z reflecte o diâmetro da árvore.

A determinação de percursos em **Grafos Eulerianos** segue critérios de paridade de graus. Num grafo não dirigido e conexo, a condição necessária e suficiente para a existência de um circuito euleriano é que todos os vértices possuam grau par. Caso existam exactamente dois vértices com grau ímpar, o grafo admite um caminho euleriano aberto. Num grafo dirigido fracamente conexo, o circuito euleriano requer que todos os nós satisfaçam $\text{grauEntrada}(u) = \text{grauSaida}(u)$.

O algoritmo de Hierholzer extrai o circuito euleriano em tempo linear $O(V + E)$. O procedimento mantém uma pilha de travessia que avança ao longo das arestas disponíveis de cada nó. Quando se atinge um vértice desprovido de arestas de saída por explorar, esse nó é transferido para a lista final de resposta e retirado da pilha. A inversão final da lista de vértices produz o circuito euleriano completo.

C++ | Algoritmo de Hierholzer

```
#include <vector>
#include <algorithm>

std::vector<std::vector<int>> AL;

std::vector<int> hierholzer(int s, int V) {
    std::vector<int> resultado, pilha, idx(V, 0);
    pilha.push_back(s);

    while (!pilha.empty()) {
        int u = pilha.back();
        if (idx[u] < (int)AL[u].size()) {
            int v = AL[u][idx[u]++];
            pilha.push_back(v);
        } else {
            resultado.push_back(u);
            pilha.pop_back();
        }
    }
    std::reverse(resultado.begin(), resultado.end());
    return resultado;
}
```

Quadro Sintético de Decisão Algorítmica em Competição

A selecção da técnica algorítmica em ambiente de prova deve ser pautada pela análise conjunta das restrições de tempo, limites de memória e propriedades estruturais do grafo descritas no enunciado:

Objectivo Estrutural	Restrições Típicas de V e E	Estrutura de Suporte	Complexidade Temporal	Algoritmo Canónico
Exploração e Conectividade	$V, E \leq 2 \times 10^5$	Vector de marcação	$O(V + E)$	Pesquisa em Profundidade (DFS).
Caminho Mínimo Não Ponderado	$V, E \leq 2 \times 10^5$	Fila FIFO	$O(V + E)$	Pesquisa em Largura (BFS).
Grelhas Matriciais 2D	$R \times C \leq 10^6$	Recursão de grelha	$O(R \cdot C)$	Preenchimento por Difusão (Flood Fill).
Dependências em Grafo Dirigido	$V, E \leq 2 \times 10^5$	Vetor de graus de entrada	$O(V + E)$	Kahn ou DFS com reversão de pilha.
Bipartição / 2-Colorabilidade	$V, E \leq 2 \times 10^5$	Vector de duas cores	$O(V + E)$	Bicoloração com BFS ou DFS.
Pontos de Articulação e Pontes	$V, E \leq 10^5$	dfs_num e dfs_low	$O(V + E)$	Algoritmo de Tarjan para cortes.
Componentes Fortemente Conexas	$V, E \leq 10^5$	Pilha explícita e dfs_low	$O(V + E)$	Algoritmo de Tarjan para SCCs.

Árvore de Cobertura Mínima	$E \leq 3 \times 10^5$	Conjuntos Disjuntos (UFDS)	$O(E \log E)$	Algoritmo de Kruskal.
Caminho Mínimo ($w \geq 0$)	$V, E \leq 2 \times 10^5$	Fila de Prioridade Mínima	$O((V + E) \log V)$	Algoritmo de Dijkstra otimizado.
Caminho Mínimo com Ciclos Negativos	$V \leq 2500, E \leq 10^4$	Lista de arestas directa	$O(V \cdot E)$	Algoritmo de Bellman-Ford.
Caminhos Entre Todos os Pares	$V \leq 450$	Matriz de Adjacência	$O(V^3)$	Algoritmo de Floyd-Warshall.
Percurso de Arestas Sem Repetição	$V, E \leq 2 \times 10^5$	Pilha e índices de aresta	$O(V + E)$	Algoritmo de Hierholzer.